

UNIVERSIDADE FEDERAL DO PARANÁ

CRISTIANO JOSE DA SILVA
FRANCISCO DE ASSIS DE LIMA FILHO
JOSÉ PRADO LEITÃO
MARLON MENDONÇA PEDERSOLI DE OLIVEIRA
PEDRO BELLE MAGALHÃES DE CASTRO
RENATO MARTINS

TRABALHO DO CURSO APRENDIZADO DE MÁQUINA
T2026

CURITIBA

2026

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Gráfico de resíduos com o melhor modelo do experimento de Admissão	9
FIGURA 2 – Gráfico de resíduos com o melhor modelo do experimento de Biomassa	10

LISTA DE TABELAS

TABELA 1 – Resultado do experimento de classificação de Veículos	4
TABELA 2 – Matriz de confusão RNA Cross Validation	4
TABELA 3 – Matriz de confusão SVM Cross Validation	4
TABELA 4 – Matriz de confusão RNA Hold out	5
TABELA 5 – Matriz de confusão SVM Hold out	5
TABELA 6 – Matriz de confusão RF Hold out	5
TABELA 7 – Matriz de confusão KNN	5
TABELA 8 – Matriz de confusão RF Cross Validation	5
TABELA 9 – Predição de novos valores para veículos com o melhor modelo .	5
TABELA 10 – Resultado do experimento de classificação de Diabetes	6
TABELA 11 – Matriz de confusão RF Hold out	6
TABELA 12 – Matriz de confusão RF Cross Validation	6
TABELA 13 – Matriz de confusão SVM Hold out	6
TABELA 14 – Matriz de confusão SVM Cross Validation	7
TABELA 15 – Matriz de confusão KNN	7
TABELA 16 – Matriz de confusão RNA Cross Validation	7
TABELA 17 – Matriz de confusão RNA Hold out	7
TABELA 18 – Predição de novos valores para diabetes com o melhor modelo .	7
TABELA 19 – Resultado do experimento de regressão de Admissão	8
TABELA 20 – Predição de novos valores para admissão com o melhor modelo .	8
TABELA 21 – Resultado do experimento de regressão de Biomassa	9
TABELA 22 – Predição de novos valores para biomassa com o melhor modelo .	9
TABELA 23 – Clusterização de veículos - 10 classes, apenas 10 primeiras ob- servações	11

SUMÁRIO

1	CLASSIFICAÇÃO	4
1.1	EXPERIMENTO COM VEÍCULOS	4
1.2	EXPERIMENTO COM DIABETES	6
2	REGRESSÃO	8
2.1	EXPERIMENTO COM ADMISSÃO	8
2.2	EXPERIMENTO COM BIOMASSA	9
3	AGRUPAMENTO	11
4	REGRAS DE ASSOCIAÇÃO	12
	APÊNDICE 1 – SCRIPT R CLASSIFICAÇÃO VEÍCULOS	13
	APÊNDICE 2 – SCRIPT R CLASSIFICAÇÃO DIABETES	24
	APÊNDICE 3 – SCRIPT R REGRESSÃO ADMISSÃO	35
	APÊNDICE 4 – SCRIPT R REGRESSÃO BIOMASSA	46
	APÊNDICE 5 – SCRIPT R AGRUPAMENTO VEICULOS	57
	APÊNDICE 6 – SCRIPT R REGRAS DE ASSOCIAÇÃO MUSCULAÇÃO	61

1 CLASSIFICAÇÃO

Seed utilizado: **202613**

Para o experimento de Classificação:

- Ordenar pela Acurácia (descendente), ou seja, a técnica de melhor acurácia ficará em primeiro na tabela.
- Após o quadro colocar:
 - Um resultado com 3 linhas com a predição de novos casos para a melhor técnica/parâmetro de maior Acurácia (criar um arquivo com novos casos à sua escolha)
 - A lista de comandos emitidos para conseguir os resultados obtidos

1.1 EXPERIMENTO COM VEÍCULOS

Técnica	Parâmetro	Acurácia
RNA Cross Validation	size=10 decay=0.1	0.86
SVM Cross-Validation	sigma=0.015 C=50	0.83
RNA Hold-out	size=30 decay=0.1	0.82
SVM Hold-out	sigma=0.02 C=100	0.8
RF Hold-out	mtry=10	0.75
KNN	k=1	0.74
RF Cross-Validation	mtry=10	0.74

TABELA 1 – Resultado do experimento de classificação de Veículos

	bus	opel	saab	van
bus	41	0	2	0
opel	2	22	17	0
saab	0	15	22	0
van	0	5	2	39

TABELA 2 – Matriz de confusão RNA Cross Validation

	bus	opel	saab	van
bus	43	0	1	0
opel	0	30	16	0
saab	0	11	25	0
van	0	1	1	39

TABELA 3 – Matriz de confusão SVM Cross Validation

	bus	opel	saab	van
bus	42	0	0	0
opel	0	32	11	0
saab	1	9	31	0
van	0	1	1	39

TABELA 4 – Matriz de confusão RNA Hold out

	bus	opel	saab	van
bus	43	0	1	0
opel	0	27	17	0
saab	0	14	24	0
van	0	1	1	39

TABELA 5 – Matriz de confusão SVM Hold out

	bus	opel	saab	van
bus	43	0	0	0
opel	0	30	16	0
saab	0	11	26	0
van	0	1	1	39

TABELA 6 – Matriz de confusão RF Hold out

	bus	opel	saab	van
bus	42	0	3	0
opel	0	24	16	0
saab	0	16	21	0
van	1	2	3	39

TABELA 7 – Matriz de confusão KNN

	bus	opel	saab	van
bus	42	0	2	0
opel	0	23	18	0
saab	0	17	20	0
van	1	2	3	39

TABELA 8 – Matriz de confusão RF Cross Validation

Comp	Circ	DCirc	RadRa	PrAx	MaxLRa	ScatRa	Elong	PrAxisR	MaxLR	ScVarM	ScVarM	RaGyr	SkewM	Skewma	Kurtma	KurtM	HolIRa	pred
100	48	83	178	72	10	162	42	20	159	176	400	184	70	6	16	187	197	van
91	40	84	141	57	9	149	45	18	143	170	330	70	72	9	14	189	199	van
92	50	106	209	66	10	207	32	23	160	223	635	220	73	14	9	188	230	van

TABELA 9 – Predição de novos valores para veículos com o melhor modelo

1.2 EXPERIMENTO COM DIABETES

Técnica	Parâmetro	Acurácia
RF Hold-out	mtry=5	0.81
RF Cross-Validation	mtry=5	0.81
SVM Hold-out	sigma=0.01 C=10	0.82
SVM Cross-Validation	sigma=0.01 C=10	0.79
KNN	k=9	0.76
RNA Cross Validation	size=30 decay=0.1	0.75
RNA Hold-out	size=30 decay=0.1	0.73

TABELA 10 – Resultado do experimento de classificação de Diabetes

	neg	pos
neg	86	22
pos	14	31

TABELA 11 – Matriz de confusão RF Hold out

	neg	pos
neg	80	21
pos	20	32

TABELA 12 – Matriz de confusão RF Cross Validation

	neg	pos
neg	80	19
pos	20	34

TABELA 13 – Matriz de confusão SVM Hold out

	neg	pos
neg	94	26
pos	6	27

TABELA 14 – Matriz de confusão SVM Cross Validation

	neg	pos
neg	94	26
pos	6	27

TABELA 15 – Matriz de confusão KNN

	neg	pos
neg	92	21
pos	8	32

TABELA 16 – Matriz de confusão RNA Cross Validation

	neg	pos
neg	91	20
pos	9	33

TABELA 17 – Matriz de confusão RNA Hold out

preg0nt	glucose	pressure	triceps	insulin	mass	pedigree	age	pred
7	91	74	0	0	18.6	0.254	31	neg
1	122	90	51	220	18.7	0.325	31	neg
1	163	72	0	0	39	1.222	33	pos

TABELA 18 – Predição de novos valores para diabetes com o melhor modelo

2 REGRESSÃO

Para o experimento de Regressão:

- Ordenar por R2 descendente, ou seja, a técnica de melhor R2 ficará em primeiro na tabela.
- Após o quadro colocar:
 - Um resultado com 3 linhas com a predição de novos casos para a melhor técnica/parâmetro de maior R2 (criar um arquivo com novos casos à sua escolha)
 - O gráfico de resíduos para a técnica/parâmetro de maior R2
 - A lista de comando emitidos no RStudio para conseguir os resultados obtidos

2.1 EXPERIMENTO COM ADMISSÃO

Técnica	Parâmetro	R2	Syx	Pearson	Rmse	MAE
RNA Hold-out	size=10 decay=0.1	0.81	0.06	0.9	0.06	0.04
RNA Cross Validation	size=10 decay=0.1	0.81	0.06	0.9	0.06	0.04
SVM Hold-out	sigma=0.01 C=10	0.8	0.06	0.9	0.06	0.04
SVM Cross-Validation	sigma=0.01 C=10	0.8	0.06	0.9	0.06	0.04
RF Hold-out	mtry=2	0.79	0.06	0.89	0.06	0.04
RF Cross-Validation	mtry=2	0.78	0.06	0.88	0.06	0.04
KNN	k=5	0.76	0.07	0.87	0.07	0.05

TABELA 19 – Resultado do experimento de regressão de Admissão

GRE.Score	TOEFL.Score	University.Rating	SOP	LOR	CGPA	Research	pred
212	118	4	4.5	4.5	7.65	1	0.47
324	58	4	4	4.5	2.87	1	0.13
200	104	3	3	3.5	8	1	0.41

TABELA 20 – Predição de novos valores para admissão com o melhor modelo

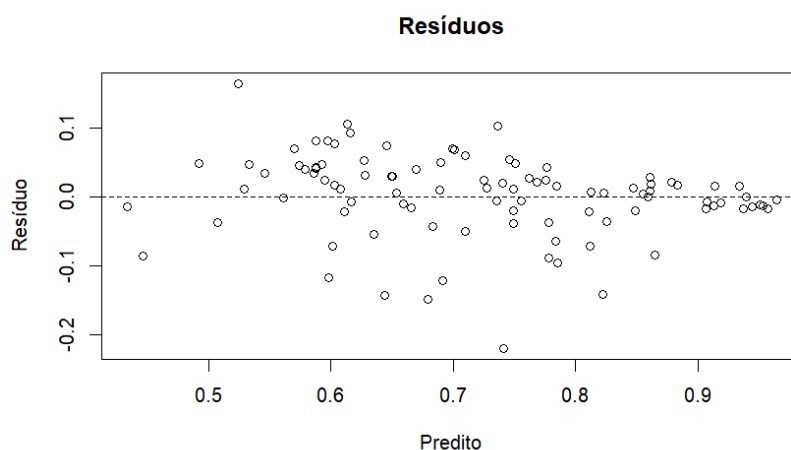


FIGURA 1 – Gráfico de resíduos com o melhor modelo do experimento de Admissão

2.2 EXPERIMENTO COM BIOMASSA

Técnica	Parâmetro	R2	Syx	Pearson	Rmse	MAE
RNA Cross Validation	size=10 decay=0.01	0.96	253.2	0.98	255.34	97.46
SVM Hold-out	sigma=0.01 C=100	0.95	158.26	0.98	157.5	89.59
SVM Cross-Validation	sigma=0.01 C=100	0.95	158.26	0.98	157.5	89.59
RF Cross-Validation	mtry=2	0.94	207.04	0.97	206.01	83.2
RF Hold-out	mtry=2	0.93	215.51	0.97	214.43	84.82
KNN	k=3	0.91	222.05	0.96	223.5	82.55
RNA Hold-out	size=30 decay=0.1	0.84	310.66	0.92	311.64	132.48

TABELA 21 – Resultado do experimento de regressão de Biomassa

dap	h	Me	pred
6.4	7	1.04	20
7.3	10	1.04	33.72
7.8	5.5	1.04	23.08
12.2	7.5	1.04	62

TABELA 22 – Predição de novos valores para biomassa com o melhor modelo

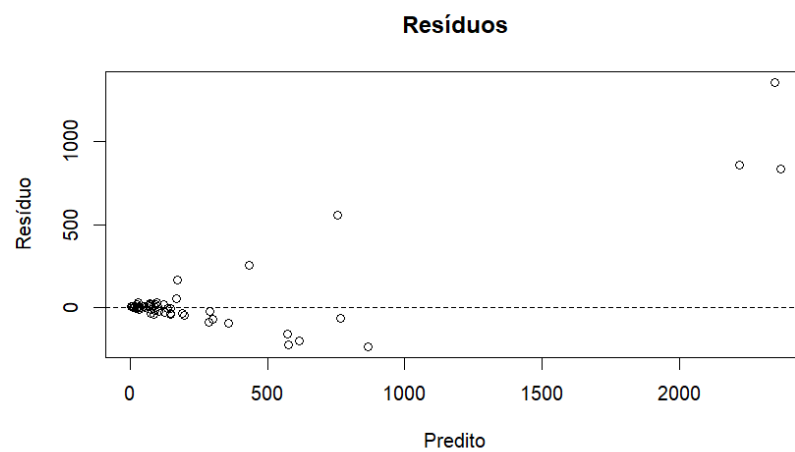


FIGURA 2 – Gráfico de resíduos com o melhor modelo do experimento de Biomassa

3 AGRUPAMENTO

Lista de Clusters gerados:

10 primeiras linhas do arquivo com o cluster correspondente. Usar 10 clusters no experimento. Colocar a lista de comandos emitidos no RStudio para conseguir os resultados obtidos

Comp	Circ	DCirc	RadRa	PrAxi	MxLR	ScatRa	Elong	PrA	MaxL	ScVarM	ScVar	RaGyr	SkewMa	Skewma	Kurtma	KurtM	HollRa	Cluster
95	48	83	178	72	10	162	42	20	159	176	379	184	70	6	16	187	197	9
91	41	84	141	57	9	149	45	19	143	170	330	158	72	9	14	189	199	4
104	50	106	209	66	10	207	32	23	158	223	635	220	73	14	9	188	196	6
93	41	82	159	63	9	144	46	19	143	160	309	127	63	6	10	199	207	7
85	44	70	205	103	52	149	45	19	144	241	325	188	127	9	11	180	183	8
107	57	106	172	50	6	255	26	28	169	280	957	264	85	5	9	181	183	2
97	43	73	173	65	6	153	42	19	143	176	361	172	66	13	1	200	204	9
90	43	66	157	65	9	137	48	18	146	162	281	164	67	3	3	193	202	7
86	34	62	140	61	7	122	54	17	127	141	223	112	64	2	14	200	208	1
93	44	98	197	62	11	183	36	22	146	202	505	152	64	4	14	195	204	3

TABELA 23 – Clusterização de veículos - 10 classes, apenas 10 primeiras observações

4 REGRAS DE ASSOCIAÇÃO

Regras geradas com uma configuração de Suporte e Confiança. Colocar a lista de comandos emitidos no RStudio para conseguir os resultados obtidos.

APÊNDICE 1 – SCRIPT R CLASSIFICAÇÃO VEÍCULOS

Trabalho-de-APM—Classificacao-Veiculos—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13     #
#                                     #
#   Classificacao Veiculos           #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA          #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSE PRADO LEITAO               #
#   MARLON MENDONCA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHAES DE CASTRO  #
#   RENATO MARTINS                  #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path::here())

### Instalacao dos pacotes necessarios
#install.packages("caret")
#install.packages("e1071")
library(caret)

## Warning: pacote 'caret' foi compilado no R versão 4.5.3

## Carregando pacotes exigidos: ggplot2
## Carregando pacotes exigidos: lattice

#####
# Macro definicoes
#####
mySEED <- 202613

DF_CSV <- "Veiculos - Dados.csv"
DF_NOVOS_CASOS <- "Veiculos - Novos Casos.csv"
CSV_NOVA_PRED <- "resultado_predicao_veiculos.csv"
DEL_COLUNA <- "a"
TARGET <- "tipo"

TARGET_TIL <- as.formula(paste(TARGET, "~ ."))
resultados <- list()
```

```

CTRL_CV <- trainControl(method = "cv", number = 5)

#####
# Funcoes locais #
#####

# Funcao para avaliar, salvar os dados
avaliar_modelo <- function(nome, modelo, teste, y_teste) {
  y_pred <- predict(modelo, teste)

  cm <- confusionMatrix(y_pred, y_teste)

  ### obtem os parametros ###
  param_str <- paste(
    names(modelo$bestTune),
    unlist(modelo$bestTune),
    sep = "=",
    collapse = " "
  )

  list_resultado <- list(
    nome = nome,
    parametro = param_str,
    modelo = modelo,
    acuracia = round(cm$overall["Accuracy"],2),
    cm = cm
  )

  #print(cm$table)

  return(list_resultado)
}

#####

### Obter os dados
dados <- read.csv(DF_CSV)
#View(dados)

### Retira coluna desnecessaria
dados[[DEL_COLUNA]] <- NULL

### Criar bases de Treino e Teste
set.seed(mySEED)
indices <- createDataPartition(dados[[TARGET]], p = 0.80, list = FALSE)
treino <- dados[indices, ]
teste <- dados[-indices, ]

y_teste <- as.factor(teste[[TARGET]])

#####
# KNN #
#####

```

```

cat("\n", "Processando KNN")

##
## Processando KNN
### Treina o modelo e faz um search grid para o valor de K

tuneGrid <- expand.grid(k = c(1, 3, 5, 7, 9))

set.seed(mySEED)

modelo <- train(
  TARGET_TIL,
  data = treino,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid
)

### Verifica o resultado do Treinamento
print(modelo)

## k-Nearest Neighbors
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## Pre-processing: centered (18), scaled (18)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 679, 679, 679, 679, 679, 679, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 1 0.6789418 0.5713737
## 3 0.6635192 0.5508808
## 5 0.6599780 0.5460807
## 7 0.6609139 0.5473572
## 9 0.6653389 0.5532572
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 1.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("KNN", modelo, teste, y_teste)

#####
# RNA Hold-out #
#####
cat("\n", "Processando RNA com Hold-out")

##
## Processando RNA com Hold-out
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(size = c(10, 30), decay = c(0.01, 0.1))

```



```

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace = FALSE
)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## Pre-processing: centered (18), scaled (18)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 679, 679, 679, 679, 679, 679, ...
## Resampling results across tuning parameters:
##
##  size  decay  Accuracy  Kappa
##  10    0.01  0.7920813  0.7221814
##  10    0.10  0.8038811  0.7380976
##  30    0.01  0.8056191  0.7404137
##  30    0.10  0.8098320  0.7459600
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 30 and decay = 0.1.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RNA Hold-out",
                                                         modelo, teste, y_teste)

#####
# RNA Cross Validation #
#####
cat("\n", "Processando RNA com Cross Validation")

##
## Processando RNA com Cross Validation

### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(size = c(10, 30), decay = c(0.01, 0.1))

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,

```

```

data = treino ,
method = "nnet" ,
preProcess = c("center", "scale"),
tuneGrid = tuneGrid ,
maxit = 300,
trace = FALSE,
trControl = CTRL_CV
)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## Pre-processing: centered (18), scaled (18)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 543, 543, 543, 544, 543
## Resampling results across tuning parameters:
##
##  size decay Accuracy Kappa
##  10  0.01  0.8011656 0.7347974
##  10  0.10  0.8159150 0.7544778
##  30  0.01  0.8144444 0.7525155
##  30  0.10  0.8144444 0.7525544
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 10 and decay = 0.1.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RNA Cross Validation",
                                                         modelo, teste, y_teste)

#####
# SVM Hold-out #
#####
cat("\n", "Processando SVM com Hold-out")

##
## Processando SVM com Hold-out

### Treina o modelo e faz um search grid para o valor de Custo e Sigma
tuneGrid = expand.grid(C = c(10, 50, 100), sigma = c(0.01, 0.015, 0.02))

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid

```

```

)

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## Pre-processing: centered (18), scaled (18)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 679, 679, 679, 679, 679, ...
## Resampling results across tuning parameters:
##
##  C      sigma  Accuracy  Kappa
##  10  0.010  0.7828979  0.7100795
##  10  0.015  0.7951169  0.7263874
##  10  0.020  0.8005652  0.7336318
##  50  0.010  0.8144443  0.7521093
##  50  0.015  0.8215949  0.7616762
##  50  0.020  0.8288166  0.7713523
## 100  0.010  0.8219239  0.7620921
## 100  0.015  0.8278773  0.7700819
## 100  0.020  0.8306867  0.7738193
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.02 and C = 100.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("SVM Hold-out", modelo,
                                                         teste, y_teste)

#####
# SVM Cross-Validation #
#####
cat("\n", "Processando SVM com Cross Validation")

##
## Processando SVM com Cross Validation
### Treina o modelo e faz um search grid para o valor de Custo e Sigma

tuneGrid = expand.grid(C = c(10, 50, 100), sigma = c(0.01, 0.015, 0.02))
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  trControl = CTRL_CV
)

```

```

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## Pre-processing: centered (18), scaled (18)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 543, 543, 543, 544, 543
## Resampling results across tuning parameters:
##
## C      sigma Accuracy  Kappa
## 10 0.010 0.7909150 0.7212077
## 10 0.015 0.8085948 0.7447592
## 10 0.020 0.8100654 0.7467527
## 50 0.010 0.8218192 0.7624116
## 50 0.015 0.8468519 0.7957591
## 50 0.020 0.8409695 0.7879129
## 100 0.010 0.8468519 0.7957612
## 100 0.015 0.8365577 0.7819881
## 100 0.020 0.8453813 0.7937915
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.015 and C = 50.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("SVM Cross-Validation",
                                                         modelo, teste, y_teste)

#####
# Random Forest Hold-out #
#####
cat("\n", "Processando Random Forest com Hold-out")

##
## Processando Random Forest com Hold-out

### Treina o modelo
set.seed(mySEED)

modelo <- train(form = TARGET_TIL ,
                 data = treino ,
                 method = "rf")

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'

```

```
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 679, 679, 679, 679, 679, 679, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.7434275  0.6575700
##   10    0.7443842  0.6587783
##   18    0.7346970  0.6458843
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 10.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RF Hold-out", modelo,
                                                         teste, y_teste)

#####
# Random Forest Cross-Validation #
#####
cat("\n", "Processando Random Forest com Cross Validation")

##
## Processando Random Forest com Cross Validation

### Treina o modelo
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "rf" ,
  trControl = CTRL_CV
)

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 679 samples
## 18 predictor
## 4 classes: 'bus', 'opel', 'saab', 'van'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 543, 543, 543, 544, 543
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.7378758  0.6505804
##   10    0.7570370  0.6760299
##   18    0.7555664  0.6740432
##
## Accuracy was used to select the optimal model using the largest value.
```

```

## The final value used for the model was mtry = 10.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RF Cross-Validation",
                                                       modelo, teste, y_teste)

#####
# Escolhe melhor modelo #
#####

### montar o ranking dos modelos ###
ranking <- data.frame(
  Modelo = sapply(resultados, function(x)
    x$nome),
  Parametro = sapply(resultados, function(x)
    x$parametro),
  Acuracia = as.numeric(sapply(resultados, function(x)
    x$acuracia))
)

ranking <- ranking[order(ranking$Acuracia, decreasing = TRUE), ]

## exibir na ordem de classificacao ##
for (i in seq_len(nrow(ranking))) {
  cat(
    i,
    "o", "\n",
    "Tecnica: ",
    ranking$Modelo[i], "\n",
    "Parametro: ",
    ranking$Parametro[i], "\n",
    "Acuracia: ",
    round(ranking$Acuracia[i], 4), "\n",
    "Matriz de Confusao:", "\n")
  print(resultados[[i]]$cm$table)
  cat('-----', "\n", "\n")
}

## 1 o
## Tecnica: RNA Cross Validation
## Parametro: size=10 decay=0.1
## Acuracia: 0.86
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  41   0   2   0
##      opel   2  22  17   0
##      saab   0  15  22   0
##      van    0   5   2  39
## -----
##
## 2 o
## Tecnica: SVM Cross-Validation

```

```

## Parametro: sigma=0.015 C=50
## Acuracia: 0.83
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  43   0   1   0
##      opel   0  30  16   0
##      saab   0  11  25   0
##      van    0   1   1  39
## -----
##
## 3 o
## Tecnica: RNA Hold-out
## Parametro: size=30 decay=0.1
## Acuracia: 0.82
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  42   0   0   0
##      opel   0  32  11   0
##      saab   1   9  31   0
##      van    0   1   1  39
## -----
##
## 4 o
## Tecnica: SVM Hold-out
## Parametro: sigma=0.02 C=100
## Acuracia: 0.8
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  43   0   1   0
##      opel   0  27  17   0
##      saab   0  14  24   0
##      van    0   1   1  39
## -----
##
## 5 o
## Tecnica: RF Hold-out
## Parametro: mtry=10
## Acuracia: 0.75
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  43   0   0   0
##      opel   0  30  16   0
##      saab   0  11  26   0
##      van    0   1   1  39
## -----
##
## 6 o
## Tecnica: KNN
## Parametro: k=1
## Acuracia: 0.74

```

```

## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  42    0    3    0
##      opel   0   24   16    0
##      saab   0   16   21    0
##      van    1    2    3   39
## -----
##
## 7 o
## Tecnica: RF Cross-Validation
## Parametro: mtry=10
## Acuracia: 0.74
## Matriz de Confusao:
##           Reference
## Prediction bus opel saab van
##      bus  42    0    2    0
##      opel   0   23   18    0
##      saab   0   17   20    0
##      van    1    2    3   39
## -----
##
## Fazer predicao com o melhor modelo ##
metricas <- sapply(resultados, function(x)
  x$acuracia)

melhor_indice <- which.max(metricas)

melhor_modelo <- resultados[[melhor_indice]]$modelo

cat("Melhor modelo:", resultados[[melhor_indice]]$nome, "\n")

## Melhor modelo: RNA Cross Validation
cat("Acuracia:", resultados[[melhor_indice]]$acuracia, "\n")

## Acuracia: 0.86
### PREDICOES DE NOVOS CASOS
dados_novos_casos <- read.csv(DF_NOVOS_CASOS)

#View(dados_novos_casos)
predict.melhor_modelo <- predict(melhor_modelo, dados_novos_casos)

resultado_melhor_modelo <- cbind(dados_novos_casos, predict.melhor_modelo)

View(resultado_melhor_modelo)

#salva em um arquivo csv
write.csv(resultado_melhor_modelo, CSV_NOVA_PRED,
  row.names=FALSE)

```


APÊNDICE 2 – SCRIPT R CLASSIFICAÇÃO DIABETES

Trabalho-de-APM—Classificacao-Diabetes—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13   #
#                                     #
#   Classificacao Diabetes         #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA        #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSE PRADO LEITAO              #
#   MARLON MENDONCA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHAES DE CASTRO  #
#   RENATO MARTINS                 #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path::here())

### Instalação dos pacotes necessários
#install.packages("caret")
#install.packages("e1071")
library(caret)

## Warning: pacote 'caret' foi compilado no R versão 4.5.3

## Carregando pacotes exigidos: ggplot2
## Carregando pacotes exigidos: lattice

#####
# Macro definições
#####
mySEED<-202613

DF_CSV<-"Diabetes - Dados.csv"
DF_NOVOS_CASOS<-"Diabetes - Novos Casos.csv"
CSV_NOVA_PRED <- "resultado_predicao_diabetes.csv"
DEL_COLUNA<-"num"
TARGET<-"diabetes"

TARGET_TIL <- as.formula(paste(TARGET, "~ ."))
resultados <- list()
```

```

CTRL_CV <- trainControl(method = "cv", number = 5)

#####
# Funcoes locais #
#####

# Funcao para avaliar, salvar os dados
avaliar_modelo <- function(nome, modelo, teste, y_teste) {
  y_pred <- predict(modelo, teste)

  cm <- confusionMatrix(y_pred, y_teste)

  ### obtem os parametros ###
  param_str <- paste(
    names(modelo$bestTune),
    unlist(modelo$bestTune),
    sep = "=",
    collapse = " "
  )

  list_resultado <- list(
    nome = nome,
    parametro = param_str,
    modelo = modelo,
    acuracia = round(cm$overall["Accuracy"],2),
    cm = cm
  )

  #print(cm$table)

  return(list_resultado)
}

#####

### Obter os dados
dados <- read.csv(DF_CSV)
#View(dados)

### Retira coluna desnecessaria
dados[[DEL_COLUNA]] <- NULL

### Criar bases de Treino e Teste
set.seed(mySEED)
indices <- createDataPartition(dados[[TARGET]], p = 0.80, list = FALSE)
treino <- dados[indices, ]
teste <- dados[-indices, ]

y_teste <- as.factor(teste[[TARGET]])

#####
# KNN #
#####

```

```

cat("\n", "Processando KNN")

##
## Processando KNN
### Treina o modelo e faz um search grid para o valor de K

tuneGrid <- expand.grid(k = c(1, 3, 5, 7, 9))

set.seed(mySEED)

modelo <- train(
  TARGET_TIL,
  data = treino,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid
)

### Verifica o resultado do Treinamento
print(modelo)

## k-Nearest Neighbors
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 615, 615, 615, 615, 615, 615, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 1 0.6882730 0.2996165
## 3 0.6890501 0.2941890
## 5 0.6942547 0.3035364
## 7 0.6962481 0.3053288
## 9 0.7063377 0.3243575
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 9.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("KNN", modelo, teste, y_teste)

#####
# RNA Hold-out #
#####
cat("\n", "Processando RNA com Hold-out")

##
## Processando RNA com Hold-out
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(size = c(10, 30), decay = c(0.01, 0.1))

```

```

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace = FALSE
)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 615, 615, 615, 615, 615, 615, ...
## Resampling results across tuning parameters:
##
##  size  decay  Accuracy  Kappa
##  10    0.01  0.6842310  0.2999815
##  10    0.10  0.7064838  0.3385178
##  30    0.01  0.6939421  0.3181139
##  30    0.10  0.7064976  0.3429285
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 30 and decay = 0.1.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RNA Hold-out",
                                                         modelo, teste, y_teste)

#####
# RNA Cross Validation #
#####
cat("\n", "Processando RNA com Cross Validation")

##
## Processando RNA com Cross Validation

### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(size = c(10, 30), decay = c(0.01, 0.1))

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,

```

```

data = treino ,
method = "nnet" ,
preProcess = c("center", "scale"),
tuneGrid = tuneGrid ,
maxit = 300,
trace = FALSE,
trControl = CTRL_CV
)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 492, 492, 492, 492, 492
## Resampling results across tuning parameters:
##
##  size decay Accuracy Kappa
##  10  0.01  0.6910569  0.3139963
##  10  0.10  0.7268293  0.3883324
##  30  0.01  0.6991870  0.3364459
##  30  0.10  0.7300813  0.3966022
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 30 and decay = 0.1.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RNA Cross Validation",
                                                         modelo, teste, y_teste)

#####
# SVM Hold-out #
#####
cat("\n", "Processando SVM com Hold-out")

##
## Processando SVM com Hold-out

### Treina o modelo e faz um search grid para o valor de Custo e Sigma
tuneGrid = expand.grid(C = c(10, 50, 100), sigma = c(0.01, 0.015, 0.02))

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid

```

```

)

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 615, 615, 615, 615, 615, 615, ...
## Resampling results across tuning parameters:
##
##  C      sigma  Accuracy  Kappa
##  10  0.010  0.7557094  0.4325193
##  10  0.015  0.7526206  0.4263186
##  10  0.020  0.7467835  0.4140491
##  50  0.010  0.7473600  0.4161192
##  50  0.015  0.7418452  0.4048290
##  50  0.020  0.7389278  0.4004419
## 100  0.010  0.7430325  0.4069472
## 100  0.015  0.7383869  0.3998862
## 100  0.020  0.7349779  0.3950834
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 10.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("SVM Hold-out", modelo,
                                                         teste, y_teste)

#####
# SVM Cross-Validation #
#####
cat("\n", "Processando SVM com Cross Validation")

##
## Processando SVM com Cross Validation
### Treina o modelo e faz um search grid para o valor de Custo e Sigma

tuneGrid = expand.grid(C = c(10, 50, 100), sigma = c(0.01, 0.015, 0.02))
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  trControl = CTRL_CV
)

```

```

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 492, 492, 492, 492, 492
## Resampling results across tuning parameters:
##
## C      sigma Accuracy  Kappa
## 10 0.010 0.7691057 0.4648035
## 10 0.015 0.7642276 0.4524095
## 10 0.020 0.7642276 0.4514640
## 50 0.010 0.7626016 0.4486850
## 50 0.015 0.7560976 0.4347634
## 50 0.020 0.7430894 0.4068621
## 100 0.010 0.7577236 0.4392192
## 100 0.015 0.7414634 0.4051185
## 100 0.020 0.7300813 0.3820846
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 10.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("SVM Cross-Validation",
                                                         modelo, teste, y_teste)

#####
# Random Forest Hold-out #
#####
cat("\n", "Processando Random Forest com Hold-out")

##
## Processando Random Forest com Hold-out

### Treina o modelo
set.seed(mySEED)

modelo <- train(form = TARGET_TIL ,
                 data = treino ,
                 method = "rf")

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'

```

```
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 615, 615, 615, 615, 615, 615, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##   2     0.7441349  0.4168642
##   5     0.7453528  0.4242807
##   8     0.7405586  0.4136448
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 5.

### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RF Hold-out", modelo,
                                                         teste, y_teste)

#####
# Random Forest Cross-Validation #
#####
cat("\n", "Processando Random Forest com Cross Validation")

##
## Processando Random Forest com Cross Validation

### Treina o modelo
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "rf" ,
  trControl = CTRL_CV
)

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 615 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 492, 492, 492, 492, 492
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##   2     0.7577236  0.4459549
##   5     0.7642276  0.4692875
##   8     0.7593496  0.4599362
##
## Accuracy was used to select the optimal model using the largest value.
```



```

## The final value used for the model was mtry = 5.
### Faz a predicao e mostra a matriz de confusao
resultados[[length(resultados) + 1]] <- avaliar_modelo("RF Cross-Validation",
                                                         modelo, teste, y_teste)

#####
# Escolhe melhor modelo #
#####

### montar o ranking dos modelos ###
ranking <- data.frame(
  Modelo = sapply(resultados, function(x)
    x$nome),
  Parametro = sapply(resultados, function(x)
    x$parametro),
  Acuracia = as.numeric(sapply(resultados, function(x)
    x$acuracia))
)

ranking <- ranking[order(ranking$Acuracia, decreasing = TRUE), ]

## exibir na ordem de classificacao ##
for (i in seq_len(nrow(ranking))) {
  cat(
    i,
    "o", "\n",
    "Tecnica: ",
    ranking$Modelo[i], "\n",
    "Parametro: ",
    ranking$Parametro[i], "\n",
    "Acuracia: ",
    round(ranking$Acuracia[i], 4), "\n",
    "Matriz de Confusao:", "\n")
  print(resultados[[i]]$cm$table)
  cat('-----', "\n", "\n")
}

## 1 o
## Tecnica: RF Hold-out
## Parametro: mtry=5
## Acuracia: 0.81
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg 86 22
##           pos 14 31
## -----
##

## 2 o
## Tecnica: RF Cross-Validation
## Parametro: mtry=5
## Acuracia: 0.81

```

```

## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg  80  21
##           pos  20  32
## -----
##
## 3 o
## Tecnica:  SVM Hold-out
## Parametro: sigma=0.01 C=10
## Acuracia:  0.79
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg  80  19
##           pos  20  34
## -----
##
## 4 o
## Tecnica:  SVM Cross-Validation
## Parametro: sigma=0.01 C=10
## Acuracia:  0.79
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg  94  26
##           pos   6  27
## -----
##
## 5 o
## Tecnica:  KNN
## Parametro: k=9
## Acuracia:  0.76
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg  94  26
##           pos   6  27
## -----
##
## 6 o
## Tecnica:  RNA Cross Validation
## Parametro: size=30 decay=0.1
## Acuracia:  0.75
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg  92  21
##           pos   8  32
## -----
##
## 7 o
## Tecnica:  RNA Hold-out
## Parametro: size=30 decay=0.1

```

```
## Acuracia: 0.73
## Matriz de Confusao:
##           Reference
## Prediction neg pos
##           neg 91 20
##           pos  9 33
## -----
##

## Fazer predicao com o melhor modelo ##
metricas <- sapply(resultados, function(x)
  x$acuracia)

melhor_indice <- which.max(metricas)

melhor_modelo <- resultados[[melhor_indice]]$modelo

cat("Melhor modelo:", resultados[[melhor_indice]]$nome, "\n")

## Melhor modelo: RF Hold-out
cat("Acuracia:", resultados[[melhor_indice]]$acuracia, "\n")

## Acuracia: 0.81
### PREDICOES DE NOVOS CASOS
dados_novos_casos <- read.csv(DF_NOVOS_CASOS)

#View(dados_novos_casos)
predict.melhor_modelo <- predict(melhor_modelo, dados_novos_casos)

resultado_melhor_modelo <- cbind(dados_novos_casos, predict.melhor_modelo)

View(resultado_melhor_modelo)

#salva em um arquivo csv
write.csv(resultado_melhor_modelo, CSV_NOVA_PRED,
  row.names=FALSE)
```

APÊNDICE 3 – SCRIPT R REGRESSÃO ADMISSÃO

Trabalho-de-APM—Regressão-Admissão—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13     #
#                                     #
#   Regressão Veiculos               #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA          #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSÉ PRADO LEITÃO                #
#   MARLON MENDONÇA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHÃES DE CASTRO  #
#   RENATO MARTINS                   #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path::here())

### Instalação dos pacotes necessários
#install.packages("caret")
#install.packages("e1071")
library(caret)

## Warning: pacote 'caret' foi compilado no R versão 4.5.3
## Carregando pacotes exigidos: ggplot2
## Carregando pacotes exigidos: lattice
### Pacote para cálculo das métricas (rmse)
#install.packages("Metrics")
library(Metrics)

## Warning: pacote 'Metrics' foi compilado no R versão 4.5.3
##
## Anexando pacote: 'Metrics'
## Os seguintes objetos são mascarados por 'package:caret':
##
##   precision, recall
#####
# Macro definições
#####
```

```

mySEED<-202613

DF_CSV<-"Admissao - Dados.csv"
DF_NOVOS_CASOS<-"Admissao - Novos Casos.csv"
CSV_NOVA_PRED <- "resultado_predicao_admissao.csv"
DEL_COLUNA<-"num"
TARGET<-"ChanceOfAdmit"

TARGET_TIL <- as.formula(paste(TARGET, "~ ."))
resultados <- list()

CTRL_CV <- trainControl(method = "cv", number = 5)

#####
# Funções locais #
#####

# Função para avaliar, salvar e mostrar metricas
avaliar_modelo <- function(nome, modelo, teste, y_teste) {

  y_pred <- predict(modelo, teste)

  # Calcula RMSE
  rmse_calc <- rmse(y_teste, y_pred)

  # Calcula R2
  r2 <- R2(y_pred, y_teste)

  ### obtem os parametros ###
  param_str <- paste(
    names(modelo$bestTune),
    unlist(modelo$bestTune),
    sep = "=",
    collapse = " "
  )

  list_resultado <- list(
    nome = nome,
    parametro = param_str,
    modelo = modelo,
    r2 = round(r2, 2),
    Syx = round(sd(y_teste - y_pred), 2),
    Pearson = round(cor(y_teste, y_pred), 2),
    rmse = round(rmse_calc, 2),
    MAE = round(mae(y_teste, y_pred), 2)
  )

  return(list_resultado)
}

#####

```

```

### Obter os dados
dados <- read.csv(DF_CSV)
#View(dados)

### Retira coluna desnecessaria
dados[[DEL_COLUNA]] <- NULL

### Criar bases de Treino e Teste
set.seed(mySEED)
indices <- createDataPartition(dados[[TARGET]], p=0.80,list=FALSE)
treino <- dados[indices,]
teste <- dados[-indices,]

y_teste <- teste[[TARGET]]

#####
# KNN #
#####
cat("\n", "Processando KNN")

##
## Processando KNN
### Treina o modelo e faz um search grid para o valor de K

tuneGrid <- expand.grid(k = c(1,3,5,7,9))

set.seed(mySEED)

modelo <- train(TARGET_TIL,
               data = treino,
               method = "knn",
               preProcess = c("center", "scale"),
               trControl = CTRL_CV,
               tuneGrid=tuneGrid)

### Verifica o resultado do Treinamento
print(modelo)

## k-Nearest Neighbors
##
## 402 samples
## 7 predictor
##
## Pre-processing: centered (7), scaled (7)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 322, 322, 321, 321, 322
## Resampling results across tuning parameters:
##
## k RMSE Rsquared MAE
## 1 0.08725184 0.6559557 0.06015741
## 3 0.06656890 0.7860378 0.04706260
## 5 0.06534430 0.7953124 0.04562049
## 7 0.06593649 0.7911925 0.04585371
## 9 0.06611204 0.7901351 0.04614873

```

```
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was k = 5.

### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("KNN", modelo, teste, y_teste)

#####
# RNA Hold-out #
#####
cat("\n", "Processando RNA com Hold-out")

##
## Processando RNA com Hold-out
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(
  size = c(10, 30),
  decay = c(0.01, 0.1)
)

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet",
  linout = TRUE,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace=FALSE)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 402 samples
## 7 predictor
##
## Pre-processing: centered (7), scaled (7)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 402, 402, 402, 402, 402, 402, ...
## Resampling results across tuning parameters:
##
##  size  decay  RMSE      Rsquared  MAE
##  10    0.01   0.07601029  0.7343533  0.05437565
##  10    0.10   0.06109377   0.8186940  0.04401757
##  30    0.01   0.08051508   0.7112857  0.05776171
##  30    0.10   0.06110606   0.8186243  0.04406580
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 10 and decay = 0.1.
```

```

### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <-avaliar_modelo("RNA Hold-out", modelo, teste, y_teste)

#####
# RNA Cross Validation #
#####
cat("\n", "Processando RNA com Cross Validation")

##
## Processando RNA com Cross Validation
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(
  size = c(10, 30),
  decay = c(0.01, 0.1)
)

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet" ,
  linout = TRUE,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace=FALSE,
  trControl = CTRL_CV)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 402 samples
## 7 predictor
##
## Pre-processing: centered (7), scaled (7)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 322, 322, 321, 321, 322
## Resampling results across tuning parameters:
##
##  size  decay  RMSE      Rsquared  MAE
##  10    0.01   0.06821047  0.7779314  0.04856783
##  10    0.10   0.06123595  0.8170863  0.04392655
##  30    0.01   0.07184446  0.7556921  0.05046249
##  30    0.10   0.06125649  0.8169680  0.04399013
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 10 and decay = 0.1.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <-avaliar_modelo("RNA Cross Validation", modelo, teste, y_teste)

```



```
#####
# SVM Hold-out #
#####
cat("\n", "Processando SVM com Hold-out")

##
## Processando SVM com Hold-out
### Treina o modelo e faz um search grid para o valor de Custo e Sigma
tuneGrid = expand.grid(C=c(10, 50, 100),
                      sigma=c(0.01, 0.015, 0.02))

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid
)

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 402 samples
## 7 predictor
##
## Pre-processing: centered (7), scaled (7)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 402, 402, 402, 402, 402, 402, ...
## Resampling results across tuning parameters:
##
##  C    sigma  RMSE      Rsquared  MAE
##  10  0.010  0.06117794  0.8193091  0.04286065
##  10  0.015  0.06180250  0.8156567  0.04334521
##  10  0.020  0.06243671  0.8118964  0.04378632
##  50  0.010  0.06264567  0.8106846  0.04401563
##  50  0.015  0.06359857  0.8042930  0.04465831
##  50  0.020  0.06445960  0.7989808  0.04530825
##  100 0.010  0.06341895  0.8055131  0.04457417
##  100 0.015  0.06456115  0.7984385  0.04540371
##  100 0.020  0.06582324  0.7907762  0.04631843
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.01 and C = 10.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("SVM Hold-out", modelo, teste, y_teste)

#####
# SVM Cross-Validation #
#####
```

```

cat("\n", "Processando SVM com Cross Validation")

##
## Processando SVM com Cross Validation
### Treina o modelo e faz um search grid para o valor de Custo e Sigma

tuneGrid = expand.grid(C=c(10, 50, 100),
                      sigma=c(0.01, 0.015, 0.02))
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  trControl = CTRL_CV)

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 402 samples
## 7 predictor
##
## Pre-processing: centered (7), scaled (7)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 322, 322, 321, 321, 322
## Resampling results across tuning parameters:
##
##  C    sigma  RMSE      Rsquared  MAE
##  10  0.010  0.06077709  0.8219186  0.04194302
##  10  0.015  0.06122517  0.8196650  0.04239960
##  10  0.020  0.06174684  0.8165364  0.04277271
##  50  0.010  0.06185041  0.8160668  0.04271986
##  50  0.015  0.06213279  0.8145354  0.04283689
##  50  0.020  0.06239144  0.8126309  0.04307468
## 100  0.010  0.06206822  0.8149345  0.04279640
## 100  0.015  0.06232620  0.8128962  0.04307208
## 100  0.020  0.06329845  0.8070790  0.04366489
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.01 and C = 10.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("SVM Cross-Validation", modelo, teste, y_teste)

#####
# Random Forest Hold-out #
#####
cat("\n", "Processando Random Forest com Hold-out")

##

```

```
## Processando Random Forest com Hold-out
### Treina o modelo
set.seed(mySEED)

tuneGrid <- expand.grid(
  mtry = c(2,3,4,5,6)
)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "rf",
  tuneGrid = tuneGrid
)

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 402 samples
## 7 predictor
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 402, 402, 402, 402, 402, 402, ...
## Resampling results across tuning parameters:
##
##  mtry  RMSE      Rsquared  MAE
##  2      0.06275822  0.8086704  0.04464434
##  3      0.06347850  0.8043573  0.04504878
##  4      0.06453355  0.7983870  0.04581344
##  5      0.06553636  0.7924422  0.04655597
##  6      0.06630404  0.7881714  0.04710941
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("RF Hold-out", modelo, teste, y_teste)

#####
# Random Forest Cross-Validation #
#####
cat("\n", "Processando Random Forest com Cross Validation")

##
## Processando Random Forest com Cross Validation
### Treina o modelo
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
```

```

method = "rf" ,
trControl = CTRL_CV
)

### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 402 samples
## 7 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 322, 322, 321, 321, 322
## Resampling results across tuning parameters:
##
## mtry RMSE Rsquared MAE
## 2 0.06150792 0.8168912 0.04324758
## 4 0.06236374 0.8110325 0.04372900
## 7 0.06382599 0.8025161 0.04497252
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.

### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("RF Cross-Validation", modelo, teste, y_teste)

#####
# Escolhe melhor modelo #
#####

### montar o ranking dos modelos ###
ranking <- data.frame(
  Modelo = sapply(resultados, function(x) x$nome),
  Parametro = sapply(resultados, function(x) x$parametro),
  R2 = sapply(resultados, function(x) x$r2),
  Syx = sapply(resultados, function(x) x$Syx),
  Pearson = sapply(resultados, function(x) x$Pearson),
  RMSE = sapply(resultados, function(x) x$rmse),
  MAE = sapply(resultados, function(x) x$MAE) )

ranking <- ranking[
  order(ranking$R2, decreasing = TRUE),
]

cat("# - Tecnica - Parametro - R2 - Syx - Pearson - RMSE - MAE", "\n")

## # - Tecnica - Parametro - R2 - Syx - Pearson - RMSE - MAE
## exibir na ordem de classificação ##
for(i in seq_len(nrow(ranking))) {
  cat(
    i, "o",

```

```

    ranking$Modelo[i],
    "_",
    ranking$Parametro[i],
    "_",
    ranking$R2[i],
    "_",
    ranking$Syx[i],
    "_",
    ranking$Pearson[i],
    "_",
    ranking$RMSE[i],
    "_",
    ranking$MAE[i],
    "\n"
  )
}

## 1 o RNA Hold-out - size=10 decay=0.1 - 0.81 - 0.06 - 0.9 - 0.06 - 0.04
## 2 o RNA Cross Validation - size=10 decay=0.1 - 0.81 - 0.06 - 0.9 - 0.06 - 0.04
## 3 o SVM Hold-out - sigma=0.01 C=10 - 0.8 - 0.06 - 0.9 - 0.06 - 0.04
## 4 o SVM Cross-Validation - sigma=0.01 C=10 - 0.8 - 0.06 - 0.9 - 0.06 - 0.04
## 5 o RF Hold-out - mtry=2 - 0.79 - 0.06 - 0.89 - 0.06 - 0.04
## 6 o RF Cross-Validation - mtry=2 - 0.78 - 0.06 - 0.88 - 0.06 - 0.04
## 7 o KNN - k=5 - 0.76 - 0.07 - 0.87 - 0.07 - 0.05

## Fazer predição com o melhor modelo ##
metricas <- sapply(resultados, function(x) x$r2)

melhor_indice <- which.max(metricas)

melhor_modelo <- resultados[[melhor_indice]]$modelo

cat("Melhor modelo:", resultados[[melhor_indice]]$nome, "\n")

## Melhor modelo: RNA Hold-out
cat("R2:", resultados[[melhor_indice]]$r2, "\n")

## R2: 0.81

### PREDIÇÕES DE NOVOS CASOS
dados_novos_casos <- read.csv(DF_NOVOS_CASOS)

#View(dados_novos_casos)
predict.melhor_modelo <- round(predict(melhor_modelo, dados_novos_casos),2)

resultado_melhor_modelo <- cbind(dados_novos_casos, predict.melhor_modelo)

View(resultado_melhor_modelo)

#salva em um arquivo csv
write.csv(resultado_melhor_modelo, CSV_NOVA_PRED,
          row.names=FALSE)

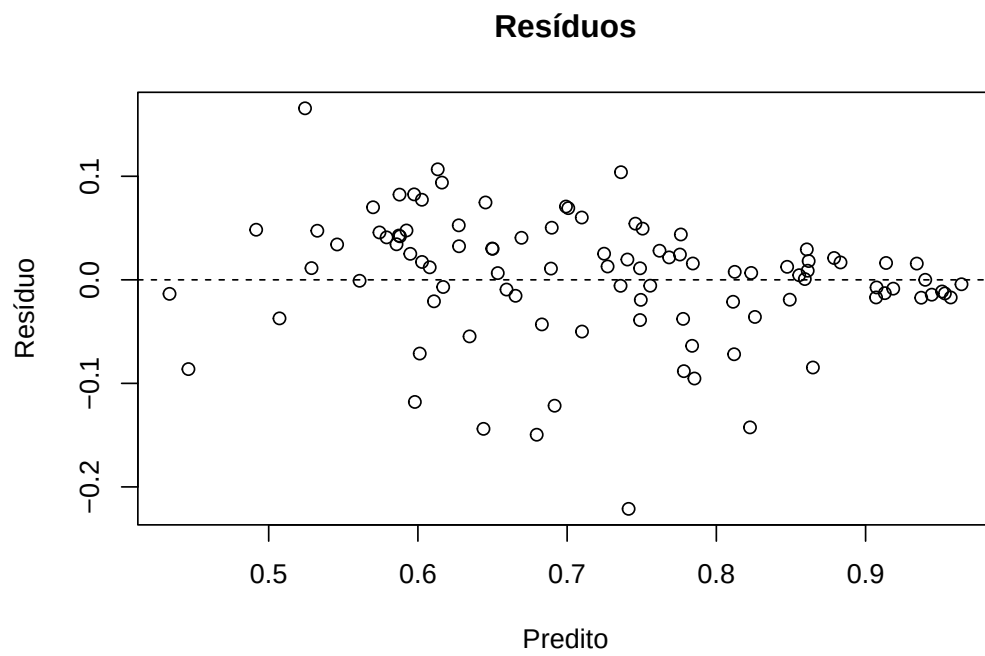
# Grafico de resíduos
y_pred = predict(melhor_modelo, newdata = teste)

```

```
residuos = y_teste - y_pred

plot(
  y_pred,
  residuos,
  xlab = "Predito",
  ylab = "Resíduo",
  main = "Resíduos"
)

abline(h = 0, lty = 2)
```



APÊNDICE 4 – SCRIPT R REGRESSÃO BIOMASSA

Trabalho-de-APM—Regressão-Biomassa—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13   #
#                                     #
#   Regressão Biomassa             #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA        #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSÉ PRADO LEITÃO              #
#   MARLON MENDONÇA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHÃES DE CASTRO  #
#   RENATO MARTINS                 #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path::here())

### Instalação dos pacotes necessários
#install.packages("caret")
#install.packages("e1071")
library(caret)

## Warning: pacote 'caret' foi compilado no R versão 4.5.3
## Carregando pacotes exigidos: ggplot2
## Carregando pacotes exigidos: lattice
### Pacote para cálculo das métricas (rmse)
#install.packages("Metrics")
library(Metrics)

## Warning: pacote 'Metrics' foi compilado no R versão 4.5.3
##
## Anexando pacote: 'Metrics'
## Os seguintes objetos são mascarados por 'package:caret':
##
##   precision, recall
#####
# Macro definições
#####
```

```

mySEED<-202613

DF_CSV<-"Biomassa - Dados.csv"
DF_NOVOS_CASOS<-"Biomassa - Novos Casos.csv"
CSV_NOVA_PRED <- "resultado_predicao_biomassa.csv"
DEL_COLUNA<-" "
TARGET<-"biomassa"

TARGET_TIL <- as.formula(paste(TARGET, "~ ."))
resultados <- list()

CTRL_CV <- trainControl(method = "cv", number = 5)

#####
# Funções locais #
#####

# Função para avaliar, salvar e mostrar metricas
avaliar_modelo <- function(nome, modelo, teste, y_teste) {

  y_pred <- predict(modelo, teste)

  # Calcula RMSE
  rmse_calc <- rmse(y_teste, y_pred)

  # Calcula R2
  r2 <- R2(y_pred, y_teste)

  ### obtem os parametros ###
  param_str <- paste(
    names(modelo$bestTune),
    unlist(modelo$bestTune),
    sep = "=",
    collapse = " "
  )

  list_resultado <- list(
    nome = nome,
    parametro = param_str,
    modelo = modelo,
    r2 = round(r2, 2),
    Syx = round(sd(y_teste - y_pred), 2),
    Pearson = round(cor(y_teste, y_pred), 2),
    rmse = round(rmse_calc,2),
    MAE = round(mae(y_teste, y_pred),2)
  )

  return(list_resultado)
}

#####

```



```

### Obter os dados
dados <- read.csv(DF_CSV)
#View(dados)

### Retira coluna desnecessaria
dados[[DEL_COLUNA]] <- NULL

### Criar bases de Treino e Teste
set.seed(mySEED)
indices <- createDataPartition(dados[[TARGET]], p=0.80,list=FALSE)
treino <- dados[indices,]
teste <- dados[-indices,]

y_teste <- teste[[TARGET]]

#####
# KNN #
#####
cat("\n", "Processando KNN")

##
## Processando KNN
### Treina o modelo e faz um search grid para o valor de K

tuneGrid <- expand.grid(k = c(1,3,5,7,9))

set.seed(mySEED)

modelo <- train(TARGET_TIL,
  data = treino,
  method = "knn",
  preProcess = c("center", "scale"),
  trControl = CTRL_CV,
  tuneGrid=tuneGrid)

### Verifica o resultado do Treinamento
print(modelo)

## k-Nearest Neighbors
##
## 240 samples
## 3 predictor
##
## Pre-processing: centered (3), scaled (3)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 192, 192, 192, 192, 192
## Resampling results across tuning parameters:
##
## k RMSE Rsquared MAE
## 1 692.3363 0.8952829 175.3978
## 3 688.0616 0.9016763 155.6465
## 5 774.5955 0.8891046 169.4767
## 7 812.5043 0.8596526 167.6406
## 9 837.4186 0.8684533 175.7808

```

```
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was k = 3.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("KNN", modelo,
                                                    teste, y_teste)

#####
# RNA Hold-out #
#####
cat("\n", "Processando RNA com Hold-out")

##
## Processando RNA com Hold-out
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(
  size = c(10, 30),
  decay = c(0.01, 0.1)
)

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet",
  linout = TRUE,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace=FALSE)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 240 samples
## 3 predictor
##
## Pre-processing: centered (3), scaled (3)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 240, 240, 240, 240, 240, 240, ...
## Resampling results across tuning parameters:
##
##  size  decay  RMSE      Rsquared  MAE
##  10    0.01  912.9165  0.8406562  213.7378
##  10    0.10  787.5216  0.8610213  185.0385
##  30    0.01  782.2065  0.8585189  197.2981
##  30    0.10  748.9119  0.8871321  193.3560
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 30 and decay = 0.1.
```

```

### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <-avaliar_modelo("RNA Hold-out", modelo,
                                                    teste, y_teste)

#####
# RNA Cross Validation #
#####
cat("\n", "Processando RNA com Cross Validation")

##
## Processando RNA com Cross Validation
### Treina o modelo e faz um search grid para o valor de size e decay
tuneGrid = expand.grid(
  size = c(10, 30),
  decay = c(0.01, 0.1)
)

set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "nnet" ,
  linout = TRUE,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  maxit = 300,
  trace=FALSE,
  trControl = CTRL_CV)

### Verifica o resultado do Treinamento
print(modelo)

## Neural Network
##
## 240 samples
## 3 predictor
##
## Pre-processing: centered (3), scaled (3)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 192, 192, 192, 192, 192
## Resampling results across tuning parameters:
##
##  size  decay  RMSE      Rsquared  MAE
##  10    0.01  493.6857  0.9230616  144.0588
##  10    0.10  781.0827  0.8795942  205.5406
##  30    0.01  828.9993  0.7590684  225.1702
##  30    0.10  555.8414  0.8905659  153.8386
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 10 and decay = 0.01.

```

```
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <-avaliar_modelo("RNA Cross Validation",
                                                    modelo, teste, y_teste)
```

```
#####
```

```
# SVM Hold-out #
```

```
#####
```

```
cat("\n", "Processando SVM com Hold-out")
```

```
##
```

```
## Processando SVM com Hold-out
```

```
### Treina o modelo e faz um search grid para o valor de Custo e Sigma
```

```
tuneGrid = expand.grid(C=c(10, 50, 100),
                       sigma=c(0.01, 0.015, 0.02))
```

```
set.seed(mySEED)
```

```
modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid
)
```

```
### Verifica o resultado do Treinamento
```

```
print(modelo)
```

```
## Support Vector Machines with Radial Basis Function Kernel
```

```
##
```

```
## 240 samples
```

```
## 3 predictor
```

```
##
```

```
## Pre-processing: centered (3), scaled (3)
```

```
## Resampling: Bootstrapped (25 reps)
```

```
## Summary of sample sizes: 240, 240, 240, 240, 240, 240, ...
```

```
## Resampling results across tuning parameters:
```

```
##
```

##	C	sigma	RMSE	Rsquared	MAE
##	10	0.010	715.1096	0.9122865	183.6101
##	10	0.015	750.3722	0.8922101	179.9110
##	10	0.020	785.7537	0.8732514	182.7515
##	50	0.010	604.1473	0.9263429	157.3799
##	50	0.015	629.9017	0.9167992	166.8051
##	50	0.020	650.9583	0.9110416	169.8314
##	100	0.010	560.4901	0.9298275	155.2240
##	100	0.015	585.8520	0.9243529	161.7674
##	100	0.020	598.3992	0.9218855	165.7862

```
##
```

```
## RMSE was used to select the optimal model using the smallest value.
```

```
## The final values used for the model were sigma = 0.01 and C = 100.
```

```
### Faz a predição e mostra a matriz de confusão
```

```
resultados[[length(resultados)+1]] <- avaliar_modelo("SVM Hold-out", modelo,
```

```

teste, y_teste)

#####
# SVM Cross-Validation #
#####
cat("\n", "Processando SVM com Cross Validation")

##
## Processando SVM com Cross Validation
### Treina o modelo e faz um search grid para o valor de Custo e Sigma

tuneGrid = expand.grid(C=c(10, 50, 100),
                      sigma=c(0.01, 0.015, 0.02))
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "svmRadial" ,
  preProcess = c("center", "scale"),
  tuneGrid = tuneGrid ,
  trControl = CTRL_CV)

### Verifica o resultado do Treinamento
print(modelo)

## Support Vector Machines with Radial Basis Function Kernel
##
## 240 samples
## 3 predictor
##
## Pre-processing: centered (3), scaled (3)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 192, 192, 192, 192, 192
## Resampling results across tuning parameters:
##
##  C      sigma  RMSE      Rsquared  MAE
##  10  0.010  539.9176  0.9330349  165.5572
##  10  0.015  537.9115  0.9282585  156.6124
##  10  0.020  557.8325  0.9225471  158.3440
##  50  0.010  437.5202  0.9372629  134.1059
##  50  0.015  442.6450  0.9404791  138.5455
##  50  0.020  448.3029  0.9412992  140.8788
## 100  0.010  401.7811  0.9408684  126.1542
## 100  0.015  403.5596  0.9420067  133.9553
## 100  0.020  404.3897  0.9438701  133.5663
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.01 and C = 100.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("SVM Cross-Validation",
                                                    modelo, teste, y_teste)

```

```
#####
# Random Forest Hold-out #
#####
cat("\n", "Processando Random Forest com Hold-out")

##
## Processando Random Forest com Hold-out
### Treina o modelo
set.seed(mySEED)

modelo <- train(
  form = TARGET_TIL ,
  data = treino ,
  method = "rf"
)

## note: only 2 unique complexity parameters in default grid. Truncating the grid to 2 .
### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 240 samples
## 3 predictor
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 240, 240, 240, 240, 240, 240, ...
## Resampling results across tuning parameters:
##
## mtry RMSE Rsquared MAE
## 2 761.8806 0.8961326 147.9267
## 3 773.0394 0.8921168 155.3156
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("RF Hold-out", modelo,
  teste, y_teste)

#####
# Random Forest Cross-Validation #
#####
cat("\n", "Processando Random Forest com Cross Validation")

##
## Processando Random Forest com Cross Validation
### Treina o modelo
set.seed(mySEED)

modelo <- train(
```

```

form = TARGET_TIL ,
data = treino ,
method = "rf" ,
trControl = CTRL_CV
)

## note: only 2 unique complexity parameters in default grid. Truncating the grid to 2 .
### Verifica o resultado do Treinamento
print(modelo)

## Random Forest
##
## 240 samples
## 3 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 192, 192, 192, 192, 192
## Resampling results across tuning parameters:
##
##  mtry  RMSE      Rsquared  MAE
##  2     522.0139  0.9452799  125.4426
##  3     531.5513  0.9365292  130.2779
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.
### Faz a predição e mostra a matriz de confusão
resultados[[length(resultados)+1]] <- avaliar_modelo("RF Cross-Validation",
                                                    modelo, teste, y_teste)

#####
# Escolhe melhor modelo #
#####

### montar o ranking dos modelos ###
ranking <- data.frame(
  Modelo = sapply(resultados, function(x) x$nome),
  Parametro = sapply(resultados, function(x) x$parametro),
  R2 = sapply(resultados, function(x) x$r2),
  Syx = sapply(resultados, function(x) x$Syx),
  Pearson = sapply(resultados, function(x) x$Pearson),
  RMSE = sapply(resultados, function(x) x$rmse),
  MAE = sapply(resultados, function(x) x$MAE) )

ranking <- ranking[
  order(ranking$R2, decreasing = TRUE),
]

cat("# - Tecnica - Parametro - R2 - Syx - Pearson - RMSE - MAE", "\n")

## # - Tecnica - Parametro - R2 - Syx - Pearson - RMSE - MAE

```

```

## exibir na ordem de classificação ##
for(i in seq_len(nrow(ranking))) {
  cat(
    i, "o",
    ranking$Modelo[i],
    "-",
    ranking$Parametro[i],
    "-",
    ranking$R2[i],
    "-",
    ranking$Syx[i],
    "-",
    ranking$Pearson[i],
    "-",
    ranking$RMSE[i],
    "-",
    ranking$MAE[i],
    "\n"
  )
}

## 1 o RNA Cross Validation - size=10 decay=0.01 - 0.96 - 253.2 - 0.98 - 255.34 - 97.46
## 2 o SVM Hold-out - sigma=0.01 C=100 - 0.95 - 158.26 - 0.98 - 157.5 - 89.59
## 3 o SVM Cross-Validation - sigma=0.01 C=100 - 0.95 - 158.26 - 0.98 - 157.5 - 89.59
## 4 o RF Cross-Validation - mtry=2 - 0.94 - 207.04 - 0.97 - 206.01 - 83.2
## 5 o RF Hold-out - mtry=2 - 0.93 - 215.51 - 0.97 - 214.43 - 84.82
## 6 o KNN - k=3 - 0.91 - 222.05 - 0.96 - 223.5 - 82.55
## 7 o RNA Hold-out - size=30 decay=0.1 - 0.84 - 310.66 - 0.92 - 311.64 - 132.48

## Fazer predição com o melhor modelo ##
metricas <- sapply(resultados, function(x) x$r2)

melhor_indice <- which.max(metricas)

melhor_modelo <- resultados[[melhor_indice]]$modelo

cat("Melhor modelo:", resultados[[melhor_indice]]$nome, "\n")

## Melhor modelo: RNA Cross Validation
cat("R2:", resultados[[melhor_indice]]$r2, "\n")

## R2: 0.96

### PREDIÇÕES DE NOVOS CASOS
dados_novos_casos <- read.csv(DF_NOVOS_CASOS)

#View(dados_novos_casos)
predict.melhor_modelo <- round(predict(melhor_modelo, dados_novos_casos), 2)

resultado_melhor_modelo <- cbind(dados_novos_casos, predict.melhor_modelo)

View(resultado_melhor_modelo)

#salva em um arquivo csv
write.csv(resultado_melhor_modelo, CSV_NOVA_PRED,

```



```

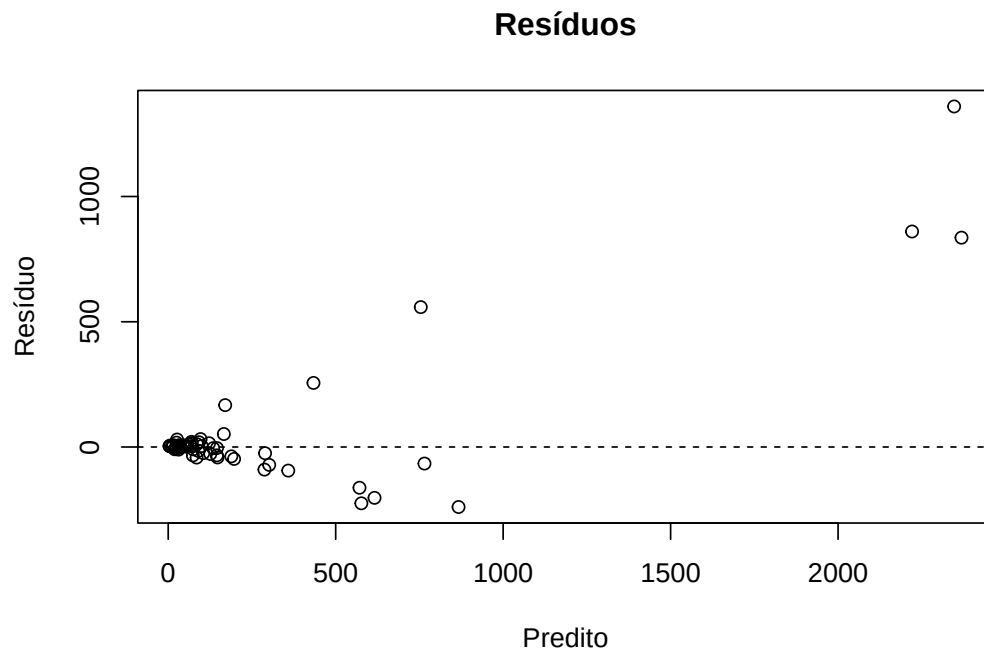
        row.names=FALSE)

# Grafico de residuos
y_pred = predict(melhor_modelo, newdata = teste)
residuos = y_teste - y_pred

plot(
  y_pred,
  residuos,
  xlab = "Predito",
  ylab = "Resíduo",
  main = "Resíduos"
)

abline(h = 0, lty = 2)

```



APÊNDICE 5 – SCRIPT R AGRUPAMENTO VEICULOS

Trabalho-de-APM—Agrupamento-Veiculos—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13     #
#                                     #
#   Agrupamento Veiculos            #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA          #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSÉ PRADO LEITÃO                #
#   MARLON MENDONÇA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHÃES DE CASTRO  #
#   RENATO MARTINS                   #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path::here())

### Instalação dos pacotes necessários
#install.packages("caret")
#install.packages("e1071")
#library(caret)

#####
# Macro definições
#####
mySEED<-202613

DF_CSV<-"Veiculos - Dados.csv"
DF_NOVOS_CASOS<-"Veiculos - Novos Casos.csv"
DEL_COLUNA<-"a"
TARGET<-"tipo"
CSV_RESULTADO <- "resultado_agrupamento.csv"

NUM_CLUSTERS <- 10

#####

### Obter os dados
```

```

dados <- read.csv(DF_CSV)
#View(dados)

### Retira coluna desnecessaria
dados[[DEL_COLUNA]] <- NULL

### Retira coluna target
dados[[TARGET]] <- NULL

#####
# Kmeans #
#####

km.res = kmeans(dados, NUM_CLUSTERS, nstart=25)

print(km.res)

## K-means clustering with 10 clusters of sizes 115, 24, 53, 164, 131, 73, 113, 8, 98, 67
##
## Cluster means:
##      Comp      Circ      DCirc      RadRa      PrAxisRa      MaxLRa      ScatRa      Elong
## 1  87.69565 37.61739 60.91304 128.0783 56.33043 6.243478 126.5826 53.27826
## 2 107.25000 55.54167 103.41667 190.2500 55.79167 5.791667 248.7083 26.87500
## 3  98.35849 47.16981 93.15094 200.0377 64.52830 8.584906 187.3019 34.98113
## 4  86.57317 44.35366 73.46341 140.2866 59.13415 8.128049 151.9329 44.44512
## 5 104.13740 53.66412 102.63359 201.8321 62.35878 10.473282 217.5573 30.64122
## 6 102.23288 50.28767 101.67123 204.4521 63.63014 9.589041 202.4932 32.58904
## 7  88.51327 40.15044 72.76991 150.7522 60.81416 7.530973 142.1947 46.84071
## 8  87.62500 45.25000 77.75000 268.1250 116.25000 49.250000 150.8750 44.62500
## 9  92.50000 41.50000 78.37755 172.7755 64.92857 7.918367 157.0714 42.05102
## 10 94.16418 42.58209 83.31343 187.2090 63.86567 7.611940 171.9104 38.07463
##      PrAxisRect MaxLRect ScVarMaxis ScVarmaxis      RaGyr SkewMaxis Skewmaxis
## 1  17.63478 131.7565 147.4348 236.7391 137.9130 73.68696 6.843478
## 2  27.08333 168.0833 271.2917 910.5417 247.1667 83.62500 6.500000
## 3  21.86792 151.5094 207.9245 529.6981 183.4528 68.47170 6.811321
## 4  19.20122 147.9085 172.1037 339.1646 176.5671 77.52439 5.652439
## 5  24.44275 169.4656 227.2901 699.3893 212.3893 71.93893 7.618321
## 6  23.15068 157.8767 219.6301 613.7123 203.9726 69.52055 6.506849
## 7  18.49558 137.6283 164.8142 301.4690 149.9115 69.53097 5.929204
## 8  19.12500 152.0000 241.2500 335.3750 181.1250 109.50000 5.000000
## 9  19.53061 140.3673 178.5918 370.1837 156.2755 68.38776 6.061224
## 10 20.56716 141.5672 194.0597 446.8955 162.8358 67.83582 5.776119
##      Kurtmaxis KurtMaxis      HollRa
## 1 11.382609 187.5043 192.1217
## 2 14.208333 182.6667 183.6250
## 3 12.660377 193.6038 199.9811
## 4  8.804878 183.2805 189.5122
## 5 16.145038 187.9160 197.4427
## 6 14.712329 190.4521 198.7534
## 7 12.353982 191.0708 197.7522
## 8 10.625000 184.6250 192.3750
## 9 12.418367 193.3163 200.0714
## 10 15.029851 194.5970 200.8806

```

```
##
## Clustering vector:
## [1] 9 4 6 7 8 2 9 7 1 3 1 1 4 4 10 6 1 10 5 6 4 1 9 7 6
## [26] 4 1 5 3 7 4 7 7 6 9 9 4 8 5 4 6 1 4 9 5 4 1 4 4 9
## [51] 7 9 5 9 2 1 3 4 6 1 3 1 1 4 3 7 4 5 7 3 5 5 10 1 4
## [76] 6 10 4 5 4 4 5 1 7 9 2 4 1 10 1 5 10 6 4 4 5 7 4 6 4
## [101] 8 9 1 1 3 5 3 7 4 3 7 4 7 1 4 9 6 5 4 10 1 4 7 1 7
## [126] 4 7 4 9 6 6 7 3 1 5 8 4 7 1 7 9 4 5 4 10 5 9 7 9 7
## [151] 3 3 7 6 7 2 7 1 7 4 1 6 10 1 5 5 9 5 4 1 5 2 7 5 4
## [176] 9 10 7 9 7 6 1 7 1 6 10 10 9 6 9 3 7 4 2 9 4 5 4 4 4
## [201] 4 9 5 5 1 4 9 4 4 5 10 7 9 5 7 9 4 5 4 1 5 4 3 4 7
## [226] 7 5 1 6 4 1 1 4 6 7 4 10 4 5 4 9 9 4 5 4 7 9 10 5 1
## [251] 1 5 4 7 4 5 9 4 6 5 1 10 1 1 3 4 4 10 1 1 4 10 7 7 2
## [276] 4 4 1 5 9 3 4 4 5 4 10 7 4 6 1 1 8 9 1 2 9 6 1 10 10
## [301] 5 4 9 1 4 4 6 5 3 6 3 4 7 6 4 1 1 3 1 5 5 2 4 3 10
## [326] 4 3 4 9 9 7 3 5 4 2 5 4 6 4 4 7 7 1 5 5 2 5 9 7 1
## [351] 3 7 4 4 10 3 4 6 9 3 5 6 10 4 4 6 4 1 4 9 10 9 9 10 1
## [376] 5 5 1 1 5 7 2 4 5 7 4 3 4 8 5 4 7 9 4 10 9 2 9 10 7
## [401] 6 9 5 7 6 9 1 7 4 10 9 1 4 7 4 5 7 9 4 1 1 6 7 4 3
## [426] 1 5 10 6 7 5 6 7 4 6 1 1 4 10 6 6 4 1 3 2 4 3 5 6 7
## [451] 9 9 9 7 6 4 7 7 5 10 1 6 10 4 3 4 4 5 5 1 7 5 5 5 4
## [476] 5 5 4 3 4 5 5 10 7 4 4 5 9 4 5 5 9 4 5 2 9 5 4 1 5
## [501] 5 5 1 7 6 5 5 7 9 5 4 9 5 9 7 1 7 5 4 7 10 7 7 8 5
## [526] 9 3 6 10 4 10 2 6 1 1 9 5 1 6 2 9 1 10 9 9 4 1 7 9 5
## [551] 1 7 10 4 6 10 2 7 4 5 5 10 5 7 9 10 6 9 4 1 5 4 9 4 6
## [576] 5 5 6 9 1 4 7 5 5 6 1 5 7 7 5 4 1 4 7 1 3 10 1 3 1
## [601] 10 7 5 9 10 5 7 1 1 4 6 4 1 9 1 7 7 1 1 5 5 4 10 5 7
## [626] 5 7 7 3 1 1 6 4 6 1 1 7 4 3 5 6 9 3 10 4 4 9 7 5 7
## [651] 6 7 9 10 10 1 4 4 7 3 1 3 4 9 7 9 3 5 9 4 3 10 6 7 9
## [676] 5 4 6 4 9 9 10 4 2 10 4 1 2 1 3 10 4 6 4 10 7 9 9 4 10
## [701] 5 6 10 7 5 5 8 1 4 10 5 6 6 5 9 5 10 10 5 5 10 5 9 3 9
## [726] 1 6 9 1 6 5 2 7 2 1 9 6 5 5 9 5 7 7 6 9 4 7 4 4 3
## [751] 10 1 9 7 1 1 3 4 4 4 3 3 7 5 5 4 10 7 6 7 4 6 3 4 1
## [776] 10 3 2 5 1 6 1 3 5 1 4 5 1 5 9 4 7 6 3 7 1 1 6 5 9
## [801] 7 1 9 10 6 4 4 6 4 7 5 7 10 4 4 4 9 5 3 3 7 5 7 6 5
## [826] 9 10 5 1 4 9 9 5 9 1 2 7 9 4 7 1 10 9 5 7 1
##
## Within cluster sum of squares by cluster:
## [1] 166756.16 115198.75 85622.79 142702.01 203281.97 113145.01 140284.74
## [8] 36338.25 136629.73 112736.84
## (between_SS / total_SS = 95.9 %)
##
## Available components:
##
## [1] "cluster" "centers" "totss" "withinss" "tot.withinss"
## [6] "betweenss" "size" "iter" "ifault"
### Associa o resultado a cada observação
resultado <- cbind(dados, km.res$cluster)

### Exibe as 10 primeiras linhas
primeiras_10 <- head(resultado, 10)
```

```
### Salva resultado das 10 primeiras linhas em um arquivo  
write.csv(primeiras_10, CSV_RESULTADO,  
          row.names=FALSE)
```

APÊNDICE 6 – SCRIPT R REGRAS DE ASSOCIAÇÃO MUSCULAÇÃO

Trabalho-de-APM—Regras-Associação-Musculação—G13-V01.R

```
#####
#                                     #
#   Trabalho de APM - T.2026 G13   #
#                                     #
#   Regras de Associação Musculação #
#                                     #
#####
#   CRISTIANO JOSE DA SILVA         #
#   FRANCISCO DE ASSIS DE LIMA FILHO #
#   JOSÉ PRADO LEITÃO               #
#   MARLON MENDONÇA PEDERSOLI DE OLIVEIRA #
#   PEDRO BELLE MAGALHÃES DE CASTRO  #
#   RENATO MARTINS                  #
#####

#install.packages("this.path")
library(this.path)

## Warning: pacote 'this.path' foi compilado no R versão 4.5.3
setwd(this.path:here())

### Instalação dos pacotes necessários
#install.packages('arules', dep=T)
library(arules)

## Warning: pacote 'arules' foi compilado no R versão 4.5.3
## Carregando pacotes exigidos: Matrix
##
## Anexando pacote: 'arules'
## Os seguintes objetos são mascarados por 'package:base':
##
##   abbreviate, write
#####
# Macro definições
#####
mySEED<-202613

DF_CSV<-"Musculacao - Dados.csv"

#####

### Obter os dados
dados <- read.transactions(file=DF_CSV,format="basket",sep=";")
```

```
## Warning in asMethod(object): removing duplicated items in transactions
### Visualiza 4 primeiras linhas
inspect(dados[1:4])

##      items
## [1] {Afundo, Crucifixo, Gemeos, LegPress}
## [2] {Agachamento, Gemeos, LegPress}
## [3] {Afundo, Agachamento, Gemeos, LegPress}
## [4] {Adutor, Agachamento, LegPress}

#####
# Apriori #
#####

### Extrai as regras
set.seed(mySEED)
rules <- apriori(dados,
                  parameter=list(supp=0.001,conf = 0.1,minlen=2),
                  appearance = list(default='rhs',lhs='LegPress'),
                  control = list(verbose=F))

inspect(sort(rules, by='confidence', decreasing=T))

##      lhs      rhs      support  confidence coverage lift
## [1] {LegPress} => {Gemeos}      0.4615385 0.5714286 0.8076923 0.8739496
## [2] {LegPress} => {Bicicleta}    0.3461538 0.4285714 0.8076923 0.7959184
## [3] {LegPress} => {Agachamento} 0.3076923 0.3809524 0.8076923 1.2380952
## [4] {LegPress} => {Esteira}      0.3076923 0.3809524 0.8076923 0.8253968
## [5] {LegPress} => {Extensor}     0.3076923 0.3809524 0.8076923 0.7619048
## [6] {LegPress} => {Afundo}       0.2692308 0.3333333 0.8076923 0.9629630
## [7] {LegPress} => {AgachamentoSmith} 0.2692308 0.3333333 0.8076923 0.8666667
## [8] {LegPress} => {Adutor}       0.1153846 0.1428571 0.8076923 1.2380952
##      count
## [1] 12
## [2] 9
## [3] 8
## [4] 8
## [5] 8
## [6] 7
## [7] 7
## [8] 3
```